M10-UF2-PR01
MÓDULO ODOO

Leandro Struni

Índice

Introducción ..... **¡Error! Marcador no definido.**

Estructura del Módulo.....3

Modelo de Datos.....5

Vistas, acciones y menús.....10

Seguridad y control de acceso.....19

Generación de tickets con código QR.....22

1.Introducción

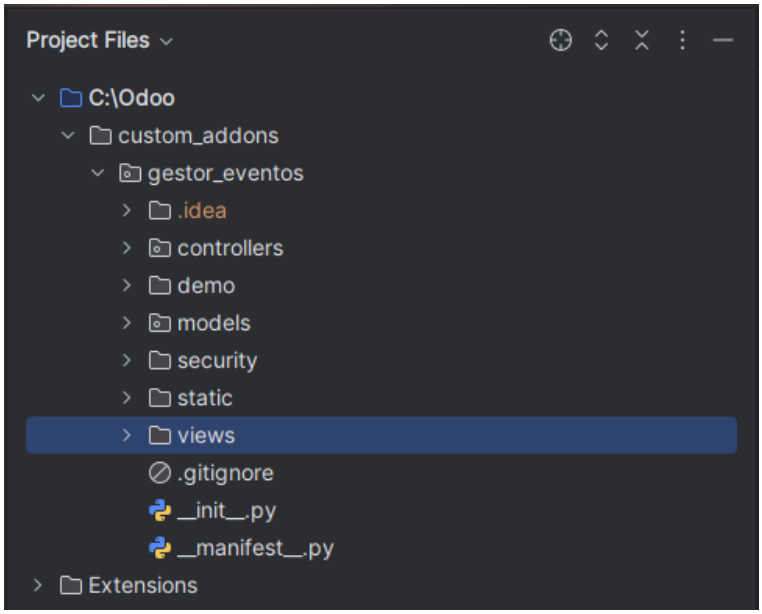
El módulo Gestor de Eventos ha sido desarrollado como parte de la práctica M10-UF2-PR01 y está destinado a facilitar la organización, gestión y control de eventos, en este caso por la asociación de vecinos de Plaça Nord.

Este módulo permite crear eventos personalizados, generar entradas con código QR, clasificar eventos mediante categorías y etiquetas, también permite gestionar ingresos y gastos asociados al evento y obtener un pequeño resumen financiero, posiblemente en un futuro se integre directamente con el módulo de finanzas de Odoo.

La finalidad del módulo es cubrir de forma integral el ciclo de vida de un evento, desde su planificación inicial hasta la generación y validación de tickets.

2. Estructura del Módulo

El módulo Gestor de Eventos sigue la estructura estándar de un módulo Odoo. Está organizado en distintas carpetas y archivos que permiten definir sus funcionalidades, vistas, permisos y apariencia.



__manifest__.py: Ficha del módulo: nombre, versión, categoría, dependencias y datos

models/: Contiene los modelos con la lógica de negocio.

views/: Define la interfaz: formularios, listas, menús, botones, etc.

security/: Define los permisos por grupo y reglas de acceso.

demo/: Contiene datos de prueba opcionales para cargar al instalar el módulo.

static/description/icon.png: Icono del módulo

Información del módulo e icono:

3. Modelo de Datos

El módulo define varios modelos personalizados que permiten estructurar y gestionar toda la información relacionada con eventos, entradas, asistentes, ingresos y gastos. A continuación, se describe cada modelo implementado:

Modelo gestor.event – Evento

Representa un evento que puede tener múltiples tickets, etiquetas y categorías.

Campo	Tipo	Descripción
name	Char	Nombre del evento (único)
description	Text	Descripción del evento
date_start / date_end	Datetime	Fechas de inicio y fin
location	Char	Lugar de realización del evento
capacity	Integer	Aforo máximo
ticket_generation_qty	Integer	Cantidad de entradas a generar con el botón personalizado
image	Binary	Imagen o cartel del evento
category_id	Many2one	Relación con una categoría (gestor.category)
tag_ids	Many2many	Relación con etiquetas (gestor.tag)
tickets	One2many	Entradas generadas (gestor.ticket)
assistants	One2many	Personas registradas al evento (gestor.assistant)
income_ids / expense_ids	One2many	Ingresos y gastos asociados
status	Selection	Estado del evento: borrador, confirmado, finalizado, etc.
total_income / total_expense / balance	Float (Compute)	Cálculo automático de finanzas del evento

Modelo gestor.ticket – Ticket/Entrada

Define las entradas o tickets que se generan para cada evento.

Genera su código único y QR en la creación.

Crea un ingreso si el ticket se marca como “Vendido”.

Permite descargar un PDF personalizado con QR.

Campo	Tipo	Descripción
ticket_type_id	Many2one	Tipo de entrada
price	Float	Precio asignado al ticket
code	Char	Código único generado automáticamente
qr_code	Binary	Código QR generado automáticamente
status	Selection	Estado de la entrada (disponible, vendida, etc.)
event_id	Many2one	Evento al que pertenece
assistant_id	Many2one	Asistente que usará la entrada (si se asocia)

Modelo gestor.tag – Etiqueta de Evento

Permite etiquetar eventos para facilitar su búsqueda o categorización.

Campo	Tipo	Descripción
name	Char	Nombre visible de la etiqueta
color	Integer	Color decorativo en la interfaz
event_ids	Many2many	Eventos asociados con la etiqueta

Modelo gestor.category – Categorí

Permite agrupar eventos por tipo.

Campo	Tipo	Descripción
name	Char	Nombre de la categoría
description	Text	Descripción (opcional)
event_ids	One2many	Eventos asociados

Modelo gestor.assitant- Asistente

Representa una persona que asiste al evento, puede tener ticket asignado.

Campo	Tipo	Descripción
name	Char	Nombre completo
email	Char	Correo electrónico
phone	Char	Teléfono
dni	Char	Documento de identidad
check_in	Boolean	Si ha asistido
check_in_time	Datetime	Fecha y hora de entrada
event_id	Many2one	Evento asignado
ticket_id	Many2one	Entrada asociada (si aplica)

Modelo gestor.income – Ingreso

Registra los ingresos del evento, manuales o por venta de tickets.

Campo	Tipo	Descripción
concept	Char	Motivo del ingreso
amount	Float	Importe ingresado
payment_date	Date	Fecha del ingreso
payment_status	Selection	Pagado o pendiente
event_id	Many2one	Evento relacionado
ticket_id	Many2one	Ticket relacionado (opcional)

Modelo gestos.expense – Gastos

Registra los gastos de organización, gastos con los proveedores, etc.

Campo	Tipo	Descripción
concept	Char	Descripción del gasto
amount	Float	Importe
date	Date	Fecha del gasto
payment_status	Selection	Pagado o pendiente
payment_date	Date	Fecha en que se pagó
notes	Text	Comentarios adicionales
event_id	Many2one	Evento relacionado
supplier_id	Many2one	Proveedor del gasto

Modelo gestor.supplier – Proveedor

Permite registrar información sobre los proveedores.

Campo	Tipo	Descripción
name	Char	Nombre del proveedor
contact_name	Char	Persona de contacto
phone	Char	Teléfono
email	Char	Email
address	Text	Dirección completa
active	Boolean	Activo o no en el sistema
expense_ids	One2many	Gastos asociados

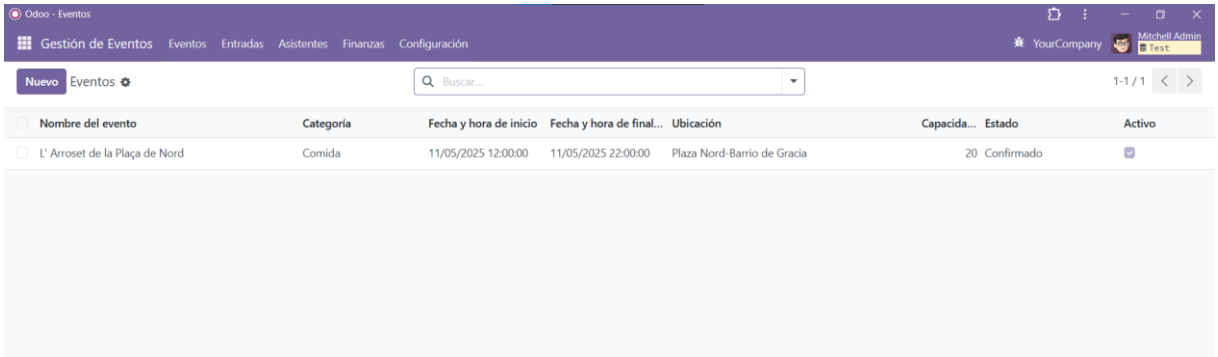
4. Vistas, acciones y menús

En este apartado se definen las vistas que permiten al usuario interactuar con los modelos creados. Se implementaron vistas de lista (tree), formulario (form) y la vinculación mediante acciones y menús personalizados.

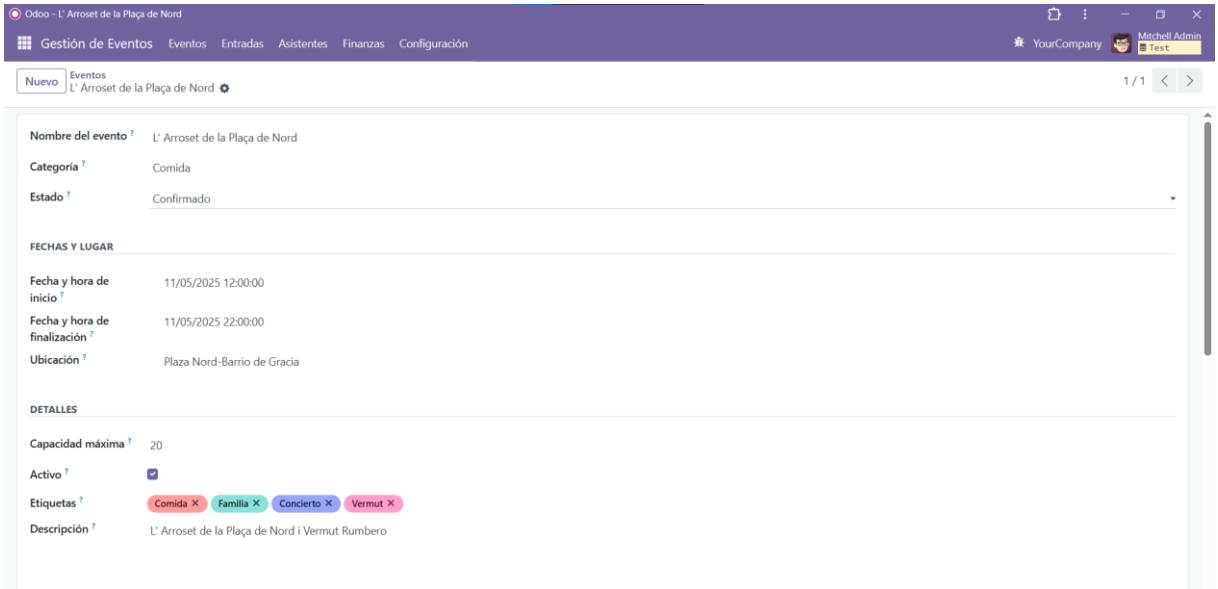
Vistas de cada modelo

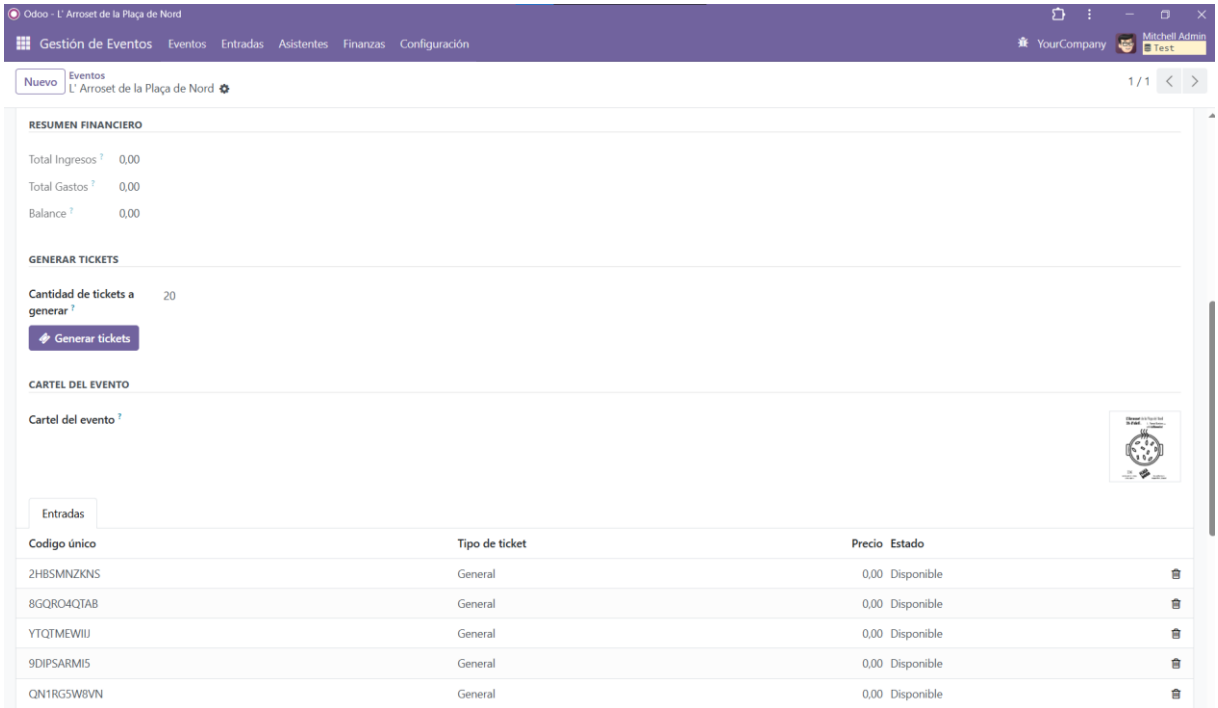
Evento (gestor.event)

- Vista de lista: Muestra los eventos creados con información como nombre, estado, fecha de inicio y finalización.



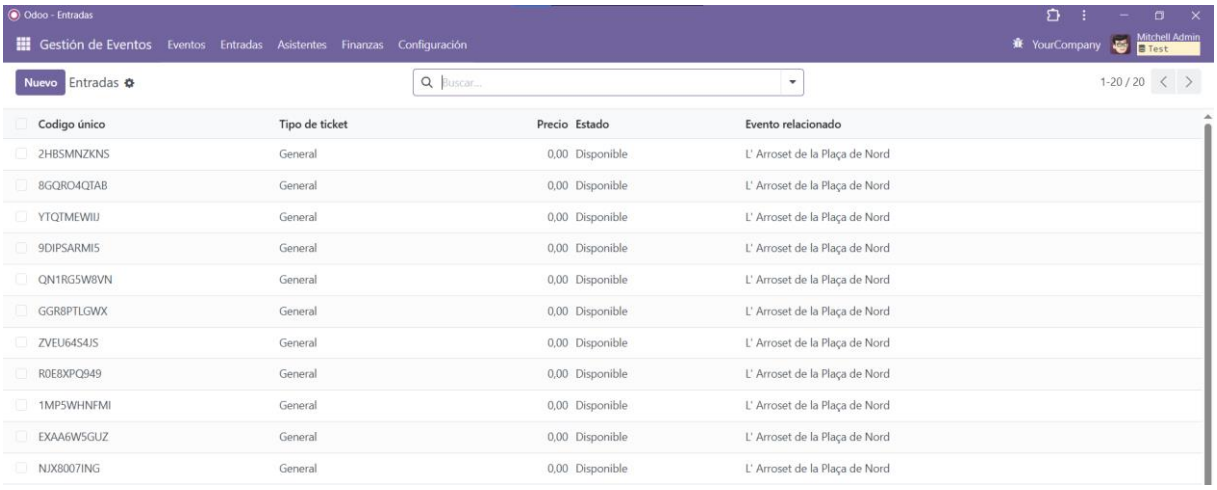
- Vista de formulario: Permite crear o editar un evento, incluyendo la imagen del cartel, etiquetas, categoría y asistentes. También incluye el botón *Generar tickets*, que nos permite generar la cantidad de tickets escogida.





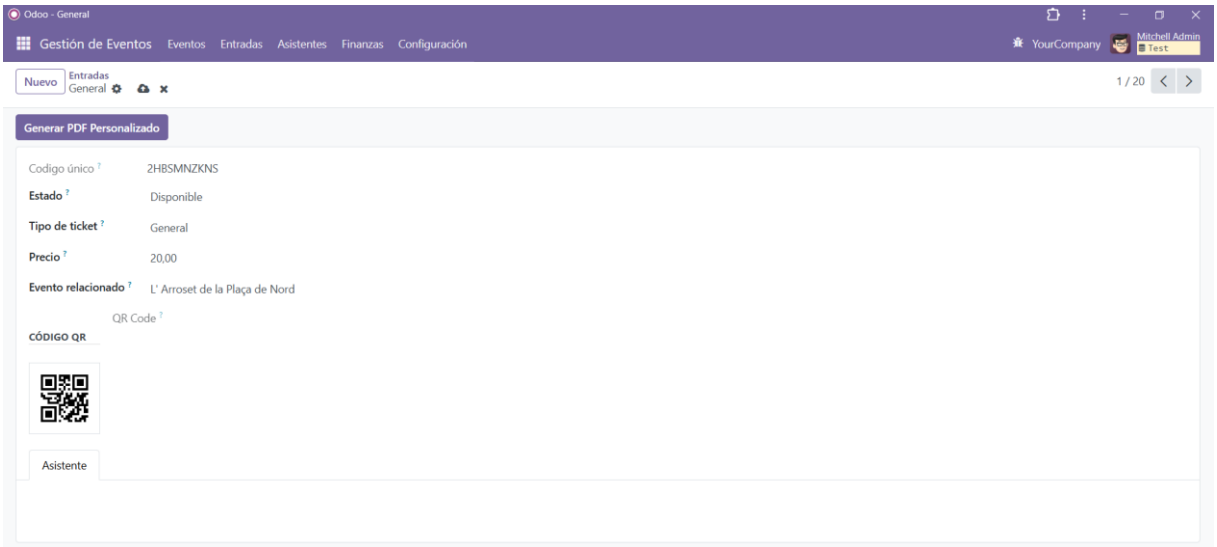
Ticket (gestor.ticket)

- Vista de lista: Visualiza el código único, tipo, precio, estado y evento relacionado de cada ticket.



Codigo único	Tipo de ticket	Precio	Estado	Evento relacionado
2HBSMNZKNS	General	0,00	Disponible	L' Arroset de la Plaça de Nord
8GQRO4QTAB	General	0,00	Disponible	L' Arroset de la Plaça de Nord
YTQTMEWIIJ	General	0,00	Disponible	L' Arroset de la Plaça de Nord
9DIPSARMI5	General	0,00	Disponible	L' Arroset de la Plaça de Nord
QN1RG5W8VN	General	0,00	Disponible	L' Arroset de la Plaça de Nord
GGR8PTLGWX	General	0,00	Disponible	L' Arroset de la Plaça de Nord
ZVEU64S4JS	General	0,00	Disponible	L' Arroset de la Plaça de Nord
R0E8XPQ949	General	0,00	Disponible	L' Arroset de la Plaça de Nord
1MP5WHNFMI	General	0,00	Disponible	L' Arroset de la Plaça de Nord
EXAA6W5GUZ	General	0,00	Disponible	L' Arroset de la Plaça de Nord
NJX8007ING	General	0,00	Disponible	L' Arroset de la Plaça de Nord

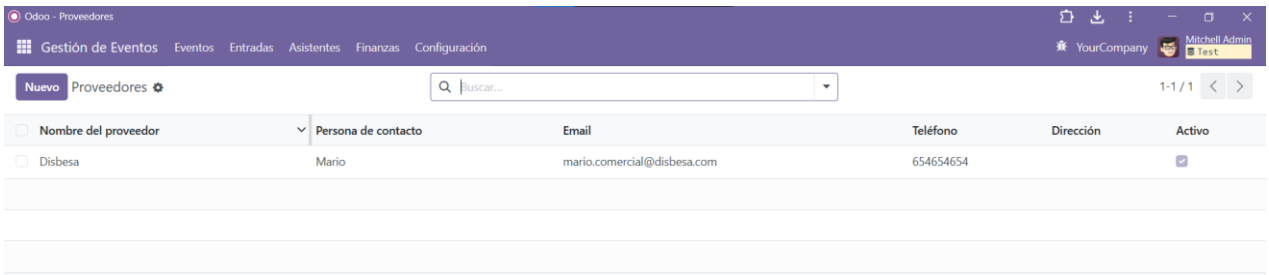
- Vista de formulario: Presenta los detalles del ticket, incluyendo el código QR generado automáticamente. También nos permite generar una entrada en PDF con los datos del evento y el código QR para permitir la validación del tickets con una App móvil, por ejemplo, y de esta forma se puede tener un mayor control sobre los tickets generados y vendidos y así evitar falsificaciones y/o duplicados.



Generar PDF con datos e imagen del evento:

Proveedor (gestor.supplier)

- Vista de lista y formulario: Permite gestionar proveedores con campos como nombre, contacto, email, teléfono y dirección.

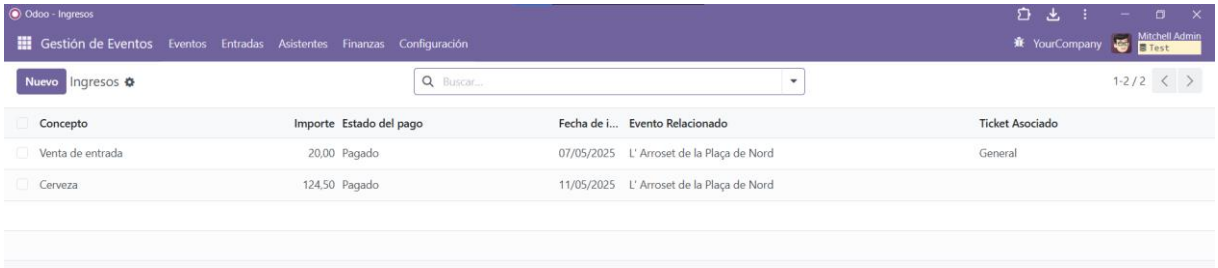


Ingresos y Gastos (gestor.income, gestor.expense)

- Vista de lista y formulario: Permite registrar movimientos económicos vinculados a eventos. Se muestran importes, fechas y descripciones.

  Cuando se marca una entrada como vendida, se realiza automáticamente el calculo y se muestra en el resumen financiero del evento.

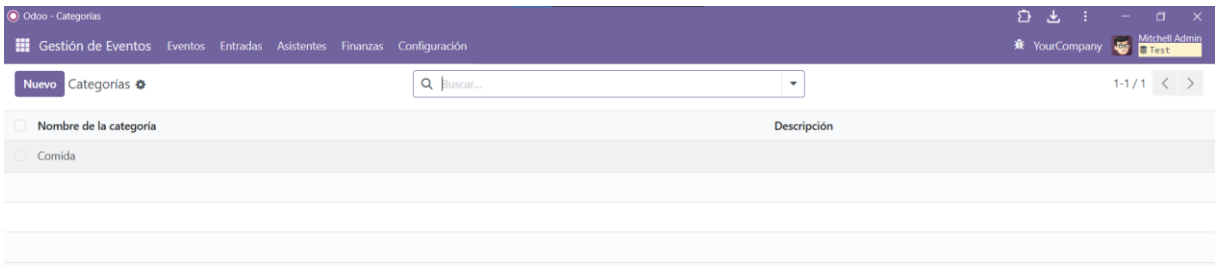
  También se puede utilizar para registrar gastos con proveedores o gastos de gestión, como también los ingresos generados por las entradas/tickets, y ventas realizadas en la barra por ejemplo.

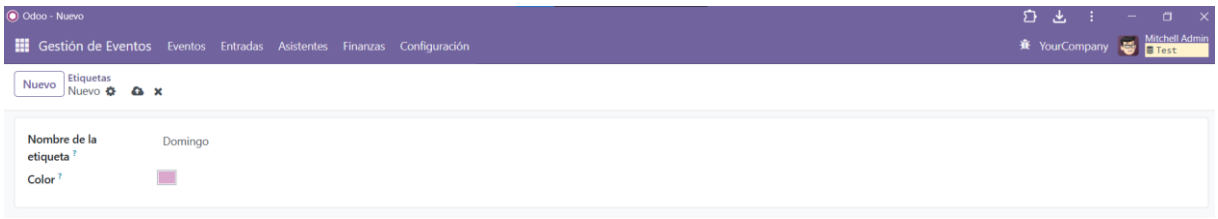


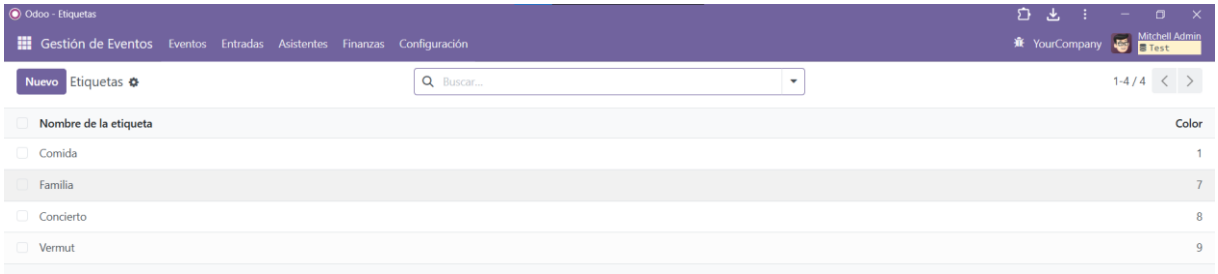
Categoría (gestor.category) y Etiqueta (gestor.tag)

- Vista de lista y formulario: Administran la categorización de eventos mediante nombres y colores.

  Permite categorizar y etiquetar los eventos para mayor control y facilitar las búsquedas mediante las etiquetas.

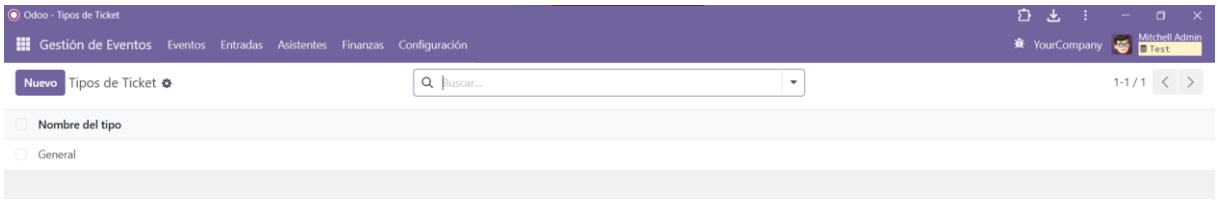






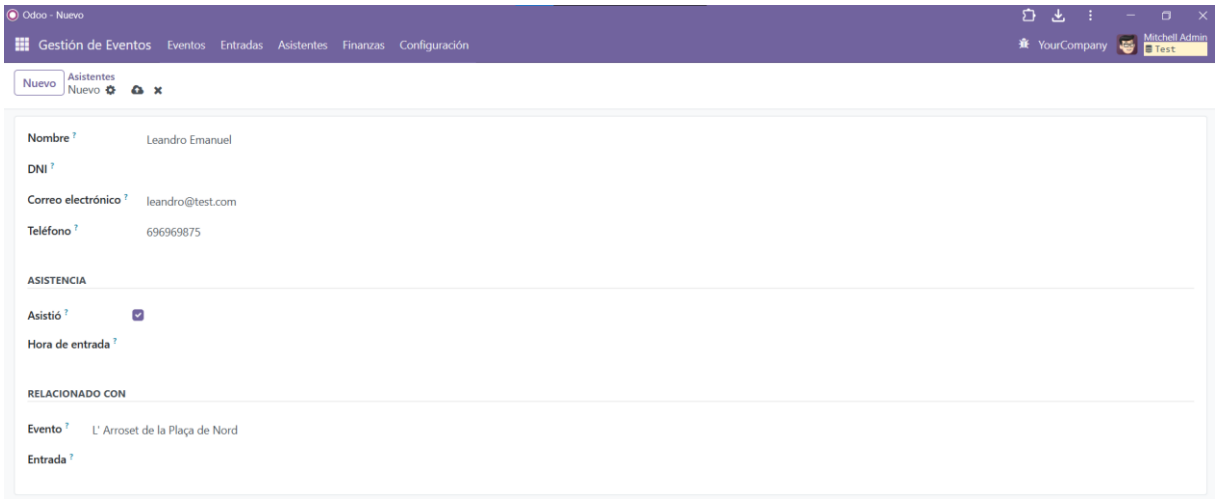
Tipo de entrada (gestor.ticket.type)

- Vista de lista y formulario: Define los tipos de ticket (ej: General, VIP), con nombre, descripción.



Asistente (gestor.assistant)

- Vista de lista y formulario: Contiene información del asistente, como nombre, DNI, email y si ha realizado check-in.



Menús y acciones

En views/menu.xml se definieron los accesos principales:

- Menú principal: “Gestión de eventos”.
- Submenús:
    - Eventos
    - Entradas
    - Tipos de entrada
    - Asistentes
    - Categorías
    - Etiquetas
    - Proveedores
    - Ingresos
    - Gastos

Cada modelo tiene asociada una acción (ir.actions.act_window) que define las vistas disponibles (list,form) y el modelo a utilizar.

Ejemplo del código para el menú y submenús:

```xml
<?xml version="1.0" encoding="UTF-8"?>
<odoo>
    <!-- Menú principal -->
    <menuitem id="menu_gestor_eventos_root"
              name="Gestión de Eventos"
              sequence="10"/>

    <!-- ======= MENÚ CONFIGURACIÓN ======= -->
    <menuitem id="menu_event_config_root"
              name="Configuración"
              parent="menu_gestor_eventos_root"
              sequence="90"/>

    <!-- Submenú Categorias -->
    <menuitem id="menu_event_category"
              name="Categorías"
              parent="menu_event_config_root"
              action="action_category_list"
              sequence="20"/>

    <!-- Submenú para etiquetas de eventos -->
    <menuitem id="menu_event_tag"
              name="Etiquetas"
              parent="menu_event_config_root"
              action="action_tag_list"
              sequence="30"/>

    <!-- Submenú para tipos de ticket -->
    <menuitem id="menu_ticket_type"
              name="Tipos de Ticket"
              parent="menu_event_config_root"
              action="action_ticket_type"
              sequence="40"/>

```

```xml
    <!-- ======= MENÚ EVENTOS ======= -->

    <!-- Menú padre de Eventos (sin acción directa) -->
    <menuitem id="menu_eventos"
              name="Eventos"
              parent="menu_gestor_eventos_root"
              sequence="10"/>

    <!-- Submenú Ver eventos -->
    <menuitem id="menu_eventos_ver"
              name="Eventos"
              parent="menu_eventos"
              action="action_event_event"
              sequence="10"/>


    <!-- ======= MENÚ ENTRADAS ======= -->

    <!-- Menú padre de Entradas (sin acción directa) -->
    <menuitem id="menu_ticket_root"
              name="Entradas"
              parent="menu_gestor_eventos_root"
              sequence="20"/>

    <!-- Submenú Ver entradas -->
    <menuitem id="menu_ticket_sub"
              name="Entradas/Tickets"
              parent="menu_ticket_root"
              action="action_ticket_list"
              sequence="10"/>
```

```xml
    <!-- ======= MENÚ ASISTENTES ======= -->

    <!-- Menú padre de Asistentes-->
    <menuitem id="menu_assistant_root"
              name="Asistentes"
              parent="menu_gestor_eventos_root"
              sequence="30"/>

    <!-- Submenú Asistentes-->
    <menuitem id="menu_assistant_sub"
              name="Asistentes"
              parent="menu_assistant_root"
              action="action_assistant_list"
              sequence="10"/>

    <!-- ======= MENÚ FINANZAS ======= -->
    <menuitem id="menu_finanzas"
              name="Finanzas"
              parent="menu_gestor_eventos_root"
              sequence="40"/>

    <!-- Submenú: Ingresos -->
    <menuitem id="menu_ingresos"
              name="Ingresos"
              parent="menu_finanzas"
              action="action_income_list"
              sequence="10"/>

    <!-- Submenú: Gastos -->
    <menuitem id="menu_gastos"
              name="Gastos"
              parent="menu_finanzas"
              action="action_expense_list"
              sequence="20"/>
```

5. Seguridad y control de acceso.

Tenemos que definir qué usuarios pueden ver, crear, editar o eliminar registros de cada modelo del módulo, diferenciando entre usuarios estándar (Event User) y administradores (Event Manager).

Definición de Grupos de Seguridad:

En el archivo security/security.xml se han creado dos grupos:

Event User: podrá ver los datos del módulo pero no puede modificarlos.

Event Manager: puede crear, leer, modificar y eliminar registros del módulo.

```
<!-- Grupos -->
<record id="group_event_user" model="res.groups">
    <field name="name">Event User</field>
</record>

<record id="group_event_manager" model="res.groups">
    <field name="name">Event Manager</field>
</record>
```

También en security.xml se ha definido cada modelo personalizado para que pueda tener configuraciones ACL asociadas, usando registros en ir.model como, por ejemplo:

Ejemplo del modelo Event:

```
<!-- Modelos -->
<record id="model_gestor_event" model="ir.model">
    <field name="name">gestor.event</field>
    <field name="model">gestor.event</field>
    <field name="state">base</field>
</record>
```

Reglas de acceso (ir.model.access.csv)

El archivo *security/ir.model.access.csv* contiene las reglas que definen qué puede hacer cada grupo sobre cada modelo.

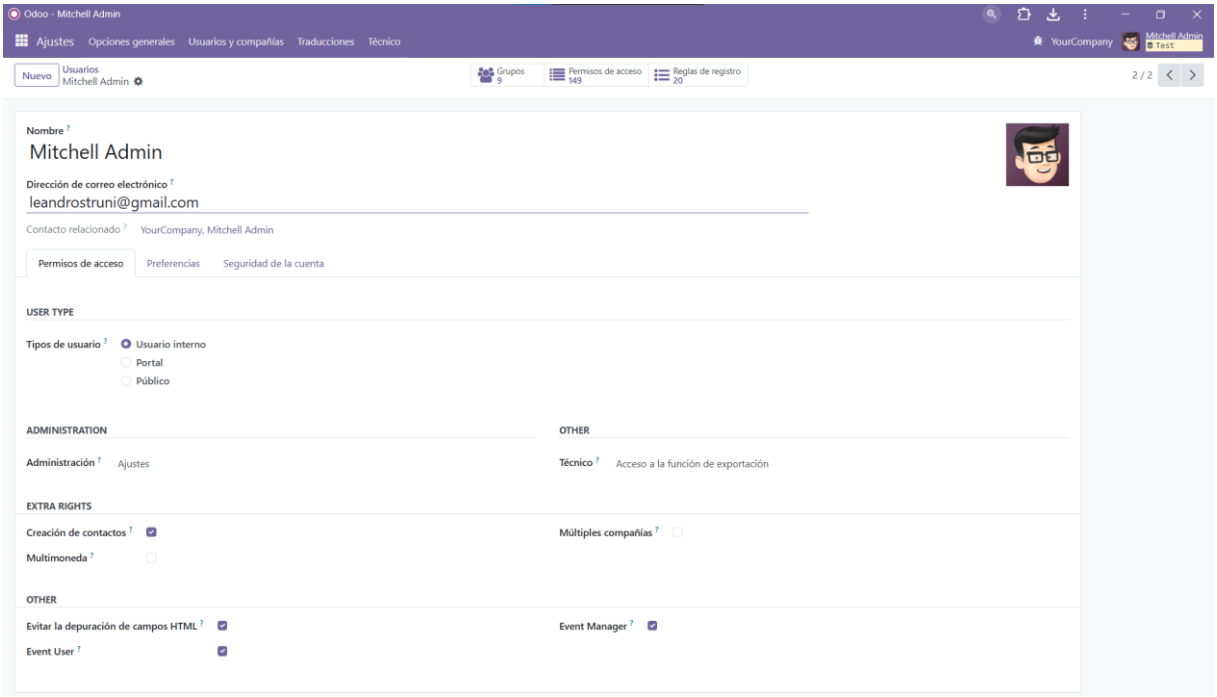
El grupo *Event User* solo puede leer porque tiene 1,0,0,0, esto aplica a todos los modelos en este caso.

El grupo *Event Manager* puede realizar todas las acciones porque está configurado con 1,1,1,1.

```csv
id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
access_event_user,access_event_user,model_gestor_event,gestor_eventos.group_event_user,1,0,0,0
access_event_manager,access_event_manager,model_gestor_event,gestor_eventos.group_event_manager,1,1,1,1

access_ticket_user,access_ticket_user,model_gestor_ticket,gestor_eventos.group_event_user,1,0,0,0
access_ticket_manager,access_ticket_manager,model_gestor_ticket,gestor_eventos.group_event_manager,1,1,1,1

access_assistant_user,access_assistant_user,model_gestor_assistant,gestor_eventos.group_event_user,1,0,0,0
access_assistant_manager,access_assistant_manager,model_gestor_assistant,gestor_eventos.group_event_manager,1,1,1,1

access_category_user,access_category_user,model_gestor_category,gestor_eventos.group_event_user,1,0,0,0
access_category_manager,access_category_manager,model_gestor_category,gestor_eventos.group_event_manager,1,1,1,1

access_tag_user,access_tag_user,model_gestor_tag,gestor_eventos.group_event_user,1,0,0,0
access_tag_manager,access_tag_manager,model_gestor_tag,gestor_eventos.group_event_manager,1,1,1,1

access_income_user,access_income_user,model_gestor_income,gestor_eventos.group_event_user,1,0,0,0
access_income_manager,access_income_manager,model_gestor_income,gestor_eventos.group_event_manager,1,1,1,1

access_expense_user,access_expense_user,model_gestor_expense,gestor_eventos.group_event_user,1,0,0,0
access_expense_manager,access_expense_manager,model_gestor_expense,gestor_eventos.group_event_manager,1,1,1,1

access_supplier_user,access_supplier_user,model_gestor_supplier,gestor_eventos.group_event_user,1,0,0,0
access_supplier_manager,access_supplier_manager,model_gestor_supplier,gestor_eventos.group_event_manager,1,1,1,1

access_gestor_ticket_type_user,access_gestor_ticket_type_user,model_gestor_ticket_type,gestor_eventos.group_event_user,1,0,0,0
access_gestor_ticket_type_manager,access_gestor_ticket_type_manager,model_gestor_ticket_type,gestor_eventos.group_event_manager,1,1,1,1
```

Para asignar los grupos al usuario hemos de ir a Ajuste->Usuarios->Usuarios y Compañías, y marcamos el check-box a los grupos que queremos asignar al usuario, es importante realizarlo ya que, aunque tengamos el módulo instalado no podremos visualizarlo si no tenemos los permisos asignados.

Como podemos ver en la captura marcamos los check-box de Event User y Event Manager


Odoo - Mitchell Admin
Ajustes
Opciones generales
Usuarios y compañías
Traducciones
Técnico
YourCompany
Mitchell Admin
Test
Nuevo
Usuarios
Mitchell Admin
Grupos 9
Permisos de acceso 149
Reglas de registro 20
2 / 2
Nombre
Mitchell Admin
Dirección de correo electrónico
leandrostruni@gmail.com
Contacto relacionado
YourCompany, Mitchell Admin
Permisos de acceso
Preferencias
Seguridad de la cuenta
USER TYPE
Tipos de usuario
Usuario interno
Portal
Público
ADMINISTRATION
Administración
Ajustes
OTHER
Técnico
Acceso a la función de exportación
EXTRA RIGHTS
Creación de contactos
Multimoneda
Múltiples compañías
OTHER
Evitar la depuración de campos HTML
Event Manager
Event User

6. Generación de tickets con código QR

Esta funcionalidad nos permite que desde la ficha de un evento se puedan generar entradas (tickets) automáticamente, cada una con un código QR único. Estos tickets se pueden luego validar escaneando el QR, y se genera también un PDF personalizado por ticket.

En la clase Event tenemos el método generate_tickets, encargado de generar los tickets:

```
def generate_tickets(self):  Leandro Struni +1
    """
    Generate a predefined number of tickets for the event.
    """
    ticket = self.env['gestor.ticket']
    try:
        ticket_type = self.env.ref('gestor_eventos.ticket_type_general', raise_if_not_found=False)
        if not ticket_type:
            ticket_type = self.env['gestor.ticket.type'].search([], limit=1)  # fallback
        for i in range(self.ticket_generation_qty):
            ticket.create({
                'event_id': self.id,
                'ticket_type_id': ticket_type.id,
                'price': 0.0,
            })
    except Exception as e:
        _logger.error( msg: "Error generating tickets for event %s: %s",  *args: self.name, str(e))
    return True
```

Y mediante los siguientes métodos, ubicados en la clase tickets, no permite generar los cogidos únicos y los códigos QR:

```python
def _generate_unique_code(self, length=10):  1 usage  Leandro Struni
    """Generate a random alphanumeric code for the ticket."""
    characters = string.ascii_uppercase + string.digits
    return ''.join(random.choices(characters, k=length))

def _generate_qr_code(self):  1 usage (1 dynamic)  Leandro Struni
    """Generate a QR code based on the ticket's unique code."""
    for record in self:
        if record.code:
            qr = qrcode.QRCode(version=1, box_size=10, border=5)
            qr.add_data(record.code)
            qr.make(fit=True)
            img = qr.make_image(fill_color="black", back_color="white")

            buffer = BytesIO()
            img.save(buffer, format='PNG')
            qr_image = base64.b64encode(buffer.getvalue())
            record.qr_code = qr_image
```

Validación de los tickets.

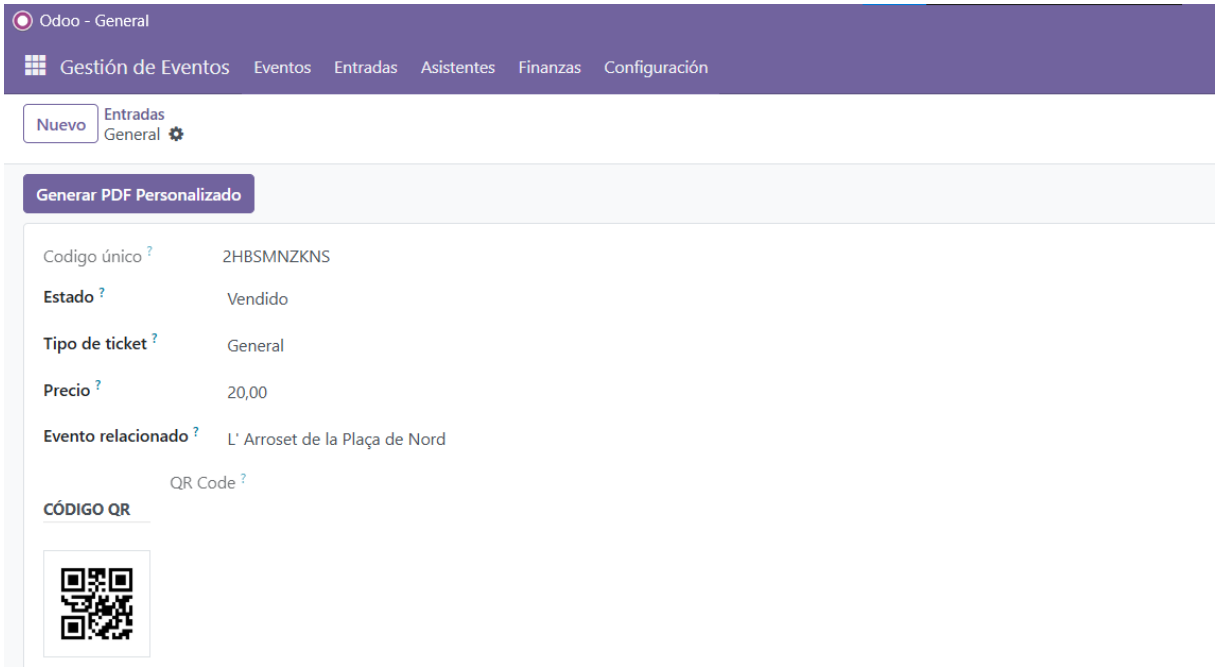
Se ha implementado un endpoint público que permite validar una entrada escaneando su código QR o ingresándolo manualmente. Este endpoint verifica que el ticket exista y esté en estado sold, y si es así, lo marca como validated.

El fichero ticket_controller.py define un controlador HTTP que expone un endpoint público. Este archivo se coloca dentro del paquete controllers del módulo. Su función principal es procesar peticiones POST con el código del ticket, buscarlo en la base de datos, verificar su estado y marcarlo como validado. En caso de errores (ticket no encontrado, ya validado, etc.) se retorna un mensaje adecuado en JSON.

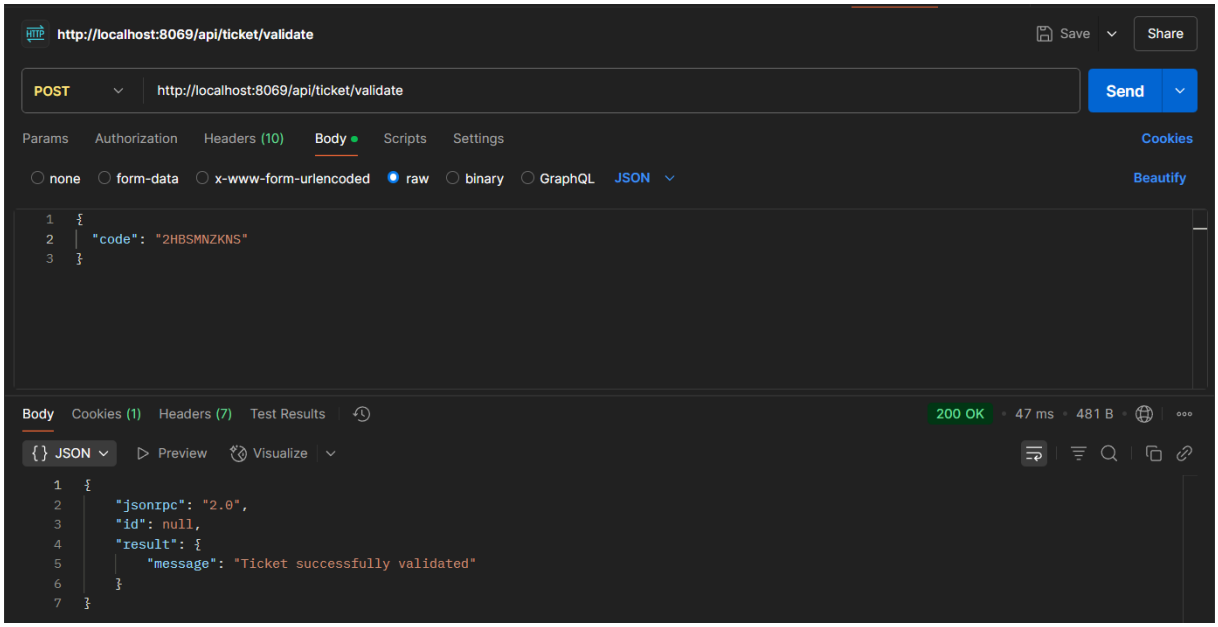
```python


class TicketController(http.Controller):  Leandro Struni

    @http.route('/api/ticket/validate', type='json', auth='public', methods=['POST'], csrf=False)  Leandro Struni
    def validate_ticket(self):
        """
        API endpoint to validate a ticket via its unique code (QR or manual input).
        It checks if the ticket exists and is in 'sold' status, then marks it as 'validated'.
        """
```

Ejemplo de uso:

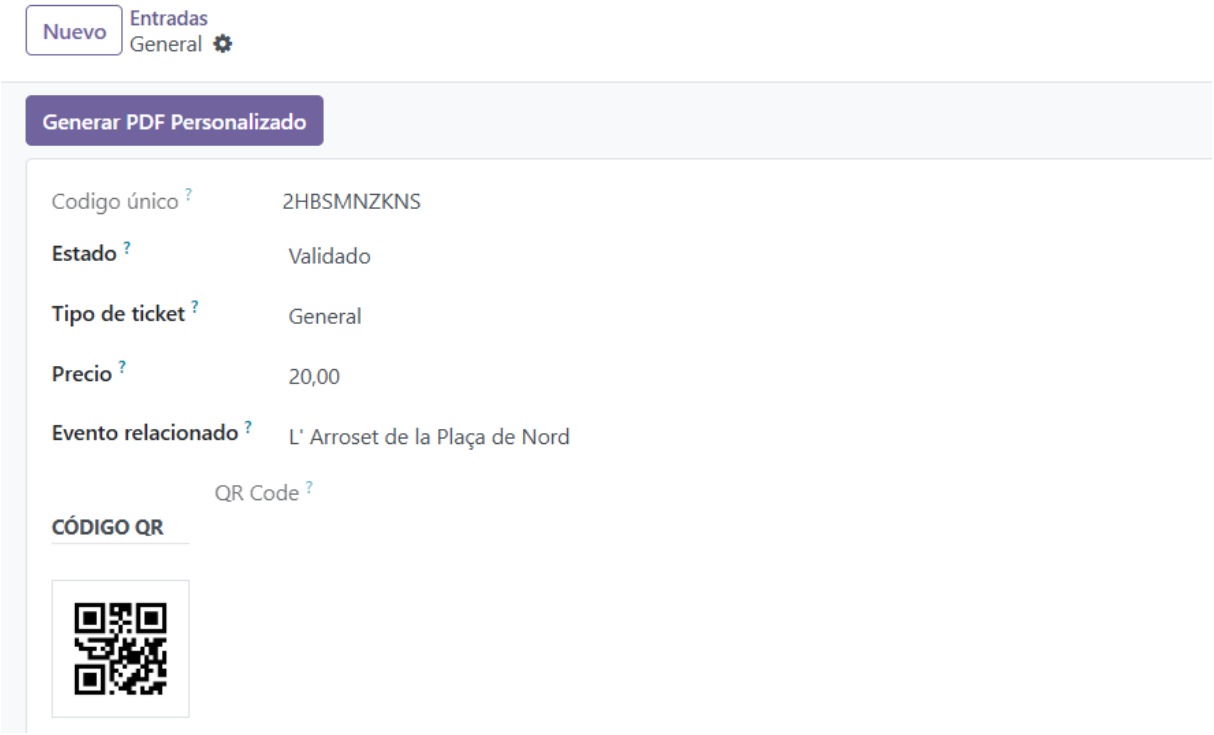
Primero debemos marcar la entrada como vendida para que pueda ser validada en el momento de consumir el ticket/entrada.



Si realizamos la llamada a la API mediante Postman, podemos verificar la respuesta y como cambia de estado a “Validated” en la interfaz de Odoo:



Cambio en la interfaz a “Validated/Validado”



SI el ticket ya fue validado anteriormente obtenemos la siguiente respuesta desde la API: